Lecture 4 - Proofs and Lemmas

Proofs are central to functional verification. We delve into Dafny's proof language and use lemmas to structure more complex proofs.

Lemmas

It may be cumbersome to verify programs by just interspersing assertions in the code; also, it is often necessary to prove non-trivial logical implications, which are not established automatically.

In such cases, Dafny provides lemmas. These are ghost objects (namely, they are just part of the specification, not of the code, hence ignored by the compiler) whose syntax resembles that of methods, but the only effect is to produce the assertions in the ensure clauses (the thesis), possibly after consuming (namely, verifying in the call context) assertions in the required clause (hypotheses):

```
lemma <lemma name>([<paramteres with types>])
    [requires <hypothesis>]
    ensures <thesis>
{
    <proof body>
}
```

To understand the meaning of a lemma, consider the schematic example of the lemma declaration

```
lemma Lemma (x: T, y: T')
    requires P x y
    ensures  Q x y
```

This declaration reads as a logical statement

```
forall x: T, y: T' :: P x y ==> Q x y
```

The proof body may be either empty when the thesis is established automatically, or a direct argument, that is, a sequence of assertions, or a case analysis, or an inductive proof.

To begin with, let us consider the following function:

```
function More(x: int): int
{
    if x <= 0 then 1
    else More(x - 2) + 3
}
```

We express with a lemma the property that x < More(x) for all integers x, namely, More is a strictly increasing function:

```
lemma Increasing(x: int)
    // no hypotheses here; if any, use requires
    ensures x < More(x)   // thesis
{} // automatic proof
```

Dafny automatically finds the proof; before making this proof explicit, let us see a possible use of the lemma while establishing an assertion in the body of a method.

```
method ExampleLemmaUse(a: int)
{
    var b := More(a); // 1.
    Increasing(a);    // 2.
    Increasing(b);    // 3.
    var c := More(b); // 4.
    assert 2 <= c - a;
}
```

The proof is obtained by calling the lemma with the parameters a and b, respectively. Here is a rephrasing of the proof of the final assertion that Dafny computes:

```
Proof of: 2 <= c - a
    5.   b == More(a)                 { by 1 }
    6.   a < More(a)                  { by 2 }
    7.   b < More(b)                  { by 3 }
    8.   a < More(a) < More(More(a))  { by 5, 6, 7 }
    9.   c == More(More(a))           { by 4, 5 }
    10   a < More(a) < c              { by 8, 9 }
    11   a + 2 <= c                   { by property of < and <= }
    12   2 <= c - a                   { by property of <= }
```

To review the proof of the lemma Increasing, we can add the instruction {:induction false} to prevent Dafny from searching for it automatically (which is only possible in some simple cases), and then write the proof by hand. First, we draft the proof as follows:

```
lemma {:induction false} IncreasingProof(x: int)
    ensures x < More(x)
{
    if x <= 0 {
        // base case: then x <= 0 < 1 == More(x) by def.
        // nothing to prove
    }
    else { // 0 < x
        IncreasingProof(x - 2); // then
        // x - 2 < More(x - 2)          { by ind. hyp. }
        // x + 1 < More(x - 2) + 3      { adding 3 to both sides }
        // x + 1 < More(x)              { def. of More }
        // x < More(x)                  { by trans. of < }
    }
}
```

This proof is accepted, but it is still implicit, performing the steps which we have guessed and written as comments. In the next proof, we let Dafny to check the single steps explicitly:

```
lemma {:induction false} IncreasingVerbose(x: int)
    ensures x < More(x)
{
    if x <= 0 {
        assert More(x) == 1; // def. of More
        assert x < More(x); // since x <= 0 < 1 == More(x)
    } else {
        assert 0 < x && More(x) == More(x - 2) + 3; // def. of More
        IncreasingVerbose(x - 2); // ind. hyp.
        assert x - 2 < More(x - 2);
        assert x + 1 < More(x - 2) + 3; // adding 3 to both sides
        assert x + 1 < More(x); // def. of More
        assert x < More(x);     // by trans. of <
    }
    assert x < More(x);
}
```

Proofs by calculations

In textbooks of mathematics, one may encounter proofs or parts of them, consisting of calculations of a series of (in)equations written:

```
a = b   by ...
  = c   by ...
  ...
  = d   by ...
```

establishing a = d by transitivity. In Dafny, we can do something similar, although with a slightly

different syntax:

```
calc {
      a
    == { <assertions or lemmas>; }
      b
    == { <assertions or lemmas>; }
    ...
    == { <assertions or lemmas>; }
      d
}
```

where those parts in {...} can be omitted when Dafny can prove the step by herself. Note that the symbol == for equality can be replaced, or interleaved with <= for less than or equal, or >= for greater than or equal (of course not both of them, which would be logically meaningless).

To see an example, consider the function:

```
function Reduce(m: nat, x: int): int
{
    if m == 0 then x
    else Reduce(m / 2, x + 1) - m
}
```

The next lemma establishes that x is an upper bound to Reduce(m, x) for any natural number m and integer x:

```
lemma {:induction false} ReduceUpperBound(m: nat, x: int)
    ensures Reduce(m, x) <= x
{
    if m == 0 {
        // trivial
        assert Reduce(m, x) == x;
    } else {
    calc { // m > 0
            Reduce(m, x);
        == // def. of Reduce
            Reduce(m / 2, x + 1) - m;
        <= { ReduceUpperBound(m / 2, x + 1); // ind. hyp.
            assert Reduce(m / 2, x + 1) <= x + 1; }
            x + 1 - m;
        <= // since m > 0
            x;
        }
    }
}
```

Similarly, the next lemma tells that x - 2 * m is a lower bound to Reduce(m, x):

```
lemma {:induction false} ReduceLowerBound(m: nat, x: int)
    ensures x - 2 * m <= Reduce(m, x)
{
    if m == 0 {
        // trivial: if m == 0 then x - 2 * m == x == Reduce(m, x)
        assert x - 2 * m <= Reduce(m, x);
    } else { // m > 0
        calc {
            Reduce(m, x);
        == // def. of Reduce
            Reduce(m / 2, x + 1) - m;
        >= { ReduceLowerBound(m / 2, x + 1);
             assert x + 1 - 2 * (m / 2) <= Reduce(m / 2, x + 1); }
            x + 1 - 2 * (m / 2) - m;
        >= { assert 2 * (m / 2) <= m; }
            // notice that 2 * (m / 2) == m holds just for even m
            // when m is odd we have 2 * (m / 2) == m - 1
            x + 1 - m - m;
        >=  x - 2 * m;
        }
    }
}
```

Remark: calc can also be used for proving chains of implications, written A ==> B, or of logical equivalences, written A <==> B. In principle, calc should work with any transitive relation.

References: based on Leino, chapter 5, paragraphs 5.0 - 5.2 and 5.4 - 5.5.